

Big Data Processing II

IA Generativa aplicada al deporte

Sistemas basados en agentes

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Marzo 2026



Universidad
Rey Juan Carlos

Mapa del bloque



Sesión 1

Control del comportamiento

Prompt Engineering + Post-training



Sesión 2

Control de la información

Retrieval-Augmented Generation (RAG)



Sesión 3

Control del flujo y la acción

Agentes, Tools y Arquitecturas Cognitivas

Introducción

Un LLM tradicional:

$$Output = f(prompt)$$

Un agente:

$$acción_{t+1} = f(estado_t, objetivo, observación_t)$$

- No solo responde.
- Decide.
- Actúa.
- Aprende del entorno.

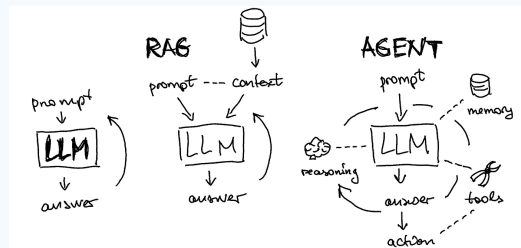


LLM + RAG permiten:

- Responder preguntas.
- Sintetizar información.

Pero no permiten:

- Ejecutar **cálculos complejos**.
- **Consultar** múltiples sistemas.
- **Tomar decisiones** secuenciales.
- **Adaptarse dinámicamente**.



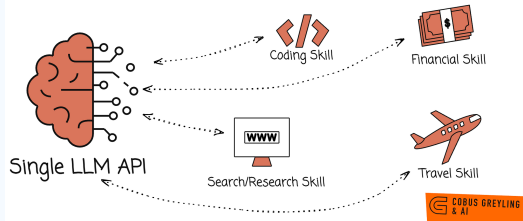
Skills y tools

Un agente se compone de **habilidades (skills)**:

- consultar bases de datos
- ejecutar modelos estadísticos
- calcular métricas avanzadas
- generar visualizaciones
- enviar alertas automáticas

Un skill es una **capacidad reutilizable**.

Anthropic Says Don't Build Agents, Build Skills Instead





Tool	Skill
Función aislada	Capacidad compuesta
Ejecuta acción	Orquesta acciones
Ej: query DB	Ej: análisis rendimiento

Ejemplos en deporte:

- API de tracking físico.
- Motor de cálculo de xG (goles esperados).
- Base histórica de rendimiento.
- Generador de dashboards.

El valor del agente crece con su ecosistema.

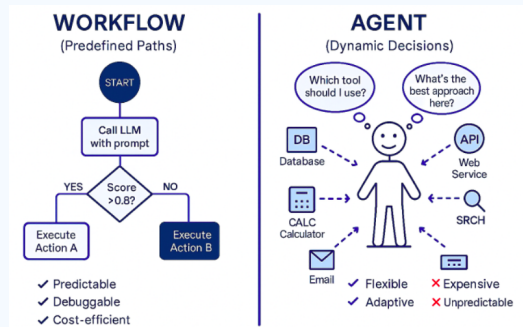


Planificación y razonamiento iterativo

Los agentes complejos no actúan en un solo paso.

Proceso:

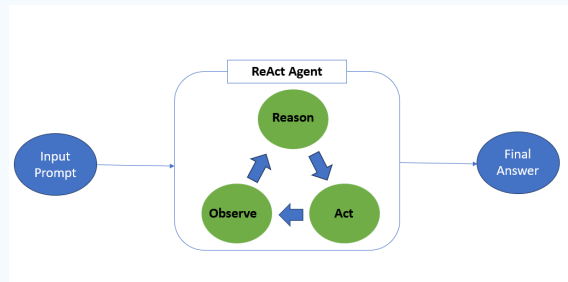
- 1 **Analizar objetivo.**
- 2 **Descomponer subtarefas.**
- 3 **Ejecutar pasos.**
- 4 **Evaluar resultados.**
- 5 **Continuar o finalizar.**



Arquitectura ReAct:

- **Pensamiento** intermedio.
- **Selección** de acción.
- **Ejecución** de tool.
- **Observación** del resultado.
- **Iteración**.

Combina **razonamiento y acción**.



Arquitectura **moderna**:

- **Planner**: diseña los pasos
- **Executor**: ejecuta herramientas
- **Evaluator**: valida resultados

Permite tareas complejas multi-etapa.



Arquitecturas dirigidas por estado (control de flujo)



Los flujos lineales (chains) fallan cuando:

- La ruta depende del contenido (distintos casos de uso).
- Hay pasos opcionales (solo si faltan datos, solo si hay anomalía).
- Hay iteración (refinar consulta, reintentar retrieval, validar).
- Se necesitan **invariantes** (no continuar si el JSON no valida).

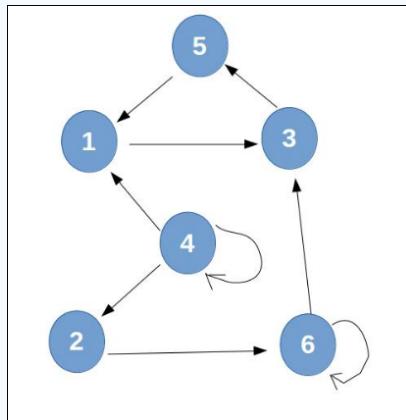
Necesidad: control explícito del flujo y de las condiciones de parada.

Una forma rigurosa de diseñar agentes es modelarlos como:

$$S \xrightarrow{\text{evento/condición}} S'$$

- **Estados** = etapas del proceso (retrieve, compute, write-report, validate).
- **Transiciones** = reglas (si falta dato → consultar DB; si inconsistente → reintentar).
- **Guardas** = condiciones que evitan errores (no generar si no hay evidencias).

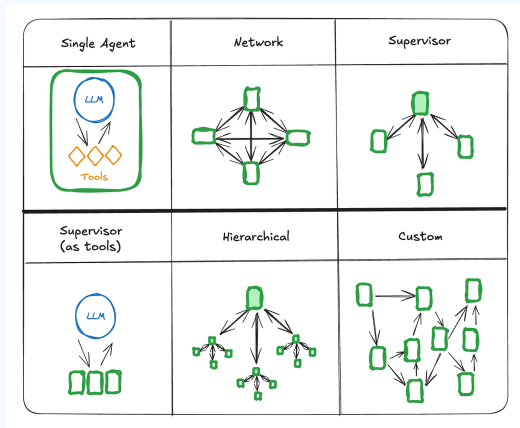
Ventaja: control del sistema > “improvisación del LLM”.



Patrones típicos en agentes:

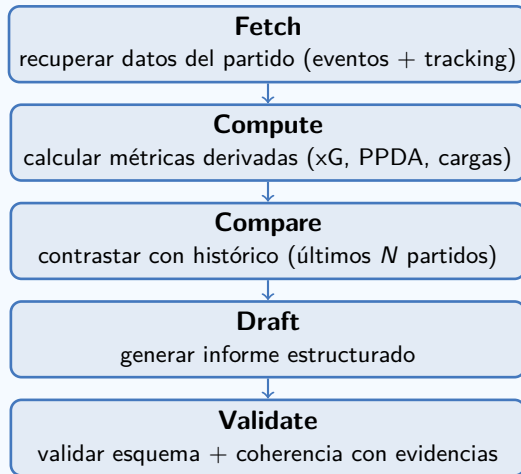
- **Router**: elegir herramienta según intención (consulta histórica vs cálculo).
- **Validator**: comprobar esquema/consistencia antes de seguir.
- **Retry with backoff**: reintentos controlados ante fallos de tool/timeout.
- **Loop bounded**: iteración con límite de pasos para evitar loops infinitos.

Idea clave: la robustez viene del control de flujo, no del “prompt perfecto”.





Estados de alto nivel:



Routing: si faltan métricas → volver a Fetch; si el JSON falla → volver a Draft.



- **Loops infinitos:** imponer límite de iteraciones y condiciones de parada.
- **Tool misuse:** validación de argumentos + esquemas estrictos.
- **Salto incorrectos:** transiciones solo si se cumplen guardas.
- **Acumulación de latencia:** cortar ramas no necesarias y cachear resultados.

Conclusión: el estado gobierna el flujo; el LLM decide dentro de límites.

Memoria y estado



Estado:

- Representación del **proceso actual** (qué sabemos ahora y qué falta).
- Vive dentro de la ejecución (por ejemplo, un diccionario/JSON de trabajo).

Memoria:

- Persistencia entre ejecuciones: historial, preferencias, perfiles, conocimiento externo.
- Vive fuera del flujo principal (DB, vector store, logs, cache).

Problema habitual: mezclar ambos genera agentes incoherentes y difíciles de depurar.

- **Conversacional (corto plazo):** historial reciente (context window).
- **Episódica:** interacciones previas resumidas (resúmenes por sesión).
- **Semántica:** documentos y conocimiento recuperable (RAG).
- **Estructurada:** perfiles, preferencias, configuraciones (tablas/JSON persistente).

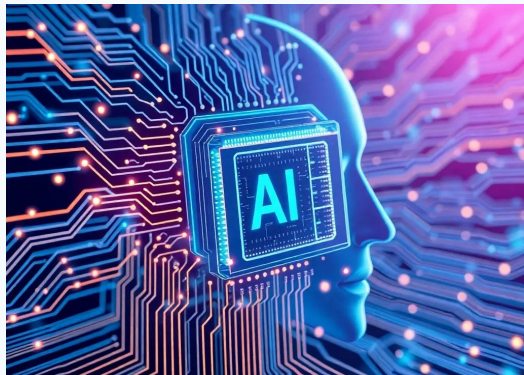
Idea clave: cada tipo tiene coste, latencia y riesgos distintos.



Preguntas que deben tener respuesta:

- **Qué se guarda** (señal vs ruido).
- **Cuándo se guarda** (al final de tarea, tras validación).
- **Cómo se recupera** (por claves, por embeddings, por filtros).
- **Caducidad** (TTL): lo viejo puede perjudicar.
- **Auditoría**: logs trazables para explicar decisiones.

Objetivo: memoria útil, no acumulación de texto.





Memoria estructurada: perfil del jugador (posición, rol, minutos, lesiones).

Memoria semántica: informes textuales de scouting y clips anotados.

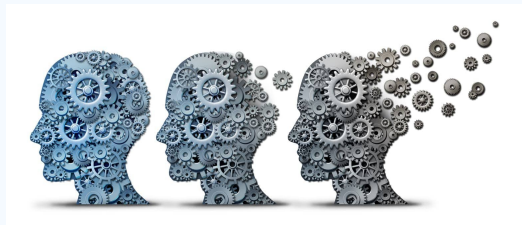
Memoria episódica: decisiones previas del agente (por qué recomendó A vs B).

Consulta:

“Dame 3 perfiles similares a este extremo, justificando con evidencias de informes.”

Resultado: respuestas consistentes porque el agente recuerda y cita.

- **Contaminación** (guardar alucinaciones)
→ guardar solo tras validación.
- **Obsolescencia** → TTL y versionado (temporada, fecha, fuente).
- **Privacidad/seguridad** → control de acceso + minimización de datos.
- **Sesgo por historiales** → resumir y normalizar, no copiar todo.



Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.

Riesgos y fallos reales en agentes

A mayor autonomía del agente:

- mayor número de componentes,
- mayor superficie de fallo,
- mayor dificultad de depuración.

Los fallos rara vez provienen del modelo aislado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.



Problemas frecuentes:

- iteraciones sin condición de parada,
- reintentos infinitos de retrieval,
- planificación redundante.

Mitigación:

- límites de iteración,
- condiciones de parada explícitas,
- control del estado.

Conclusión: la memoria aumenta capacidad, pero sin control degrada fiabilidad.



El agente puede:

- invocar tools equivocadas,
- usar parámetros inválidos,
- interpretar mal los resultados.

Mitigación:

- esquemas de entrada estrictos,
- validación automática,
- manejo robusto de errores.



Riesgo crítico:

- el sistema produce resultados plausibles pero incorrectos.

Soluciones:

- Logging detallado,
- Trazabilidad de evidencias,
- Auditoría de decisiones.



Sin trazabilidad, el sistema no es confiable.

Los agentes pueden:

- ejecutar múltiples herramientas,
- generar grandes cantidades de tokens,
- realizar múltiples iteraciones.

Resultado:

- latencias elevadas,
- costes operativos altos.

Optimizar flujo es esencial.



Si los datos son incorrectos:

- el agente amplifica errores,
- las conclusiones pueden ser erróneas,
- aumenta el riesgo en decisiones deportivas.

Regla: garbage in → garbage out.



- Validar entradas y salidas.
- Limitar iteraciones y costes.
- Registrar decisiones y evidencias.
- Diseñar flujos controlados por estado.
- Mantener supervisión humana.

Conclusión: robustez > autonomía.



Conclusiones



1. Un agente no es un LLM con más prompts

- Es un sistema que percibe, decide y actúa.
- El LLM es el motor de decisión, no el sistema completo.

2. El valor real surge de integrar herramientas y datos

- Los agentes delegan cálculos y consultas a tools externas.
- La precisión proviene de combinar IA generativa y procesos deterministas.

3. El control del flujo y del estado determina la robustez

- Arquitecturas dirigidas por estado evitan comportamientos erráticos.
- Sin control del flujo aparecen loops, errores y costes excesivos.

4. La memoria y la trazabilidad permiten sistemas fiables

- Registrar evidencias y decisiones es esencial para confiar en el sistema.

Conclusión: los agentes no añaden “inteligencia mágica”; permiten construir sistemas que toman decisiones útiles en entornos reales.